

DirectoryInfo Class in C# with Examples

In this article, I will discuss **DirectoryInfo Class in C#** with Examples. Please read our previous article discussing **FileInfo Class in C#** with Examples. C# DirectoryInfo class provides functionality to work with folders or directories. At the end of this article, you will understand what DirectoryInfo Class is in C# and when and how to use DirectoryInfo Class in C# with Examples.

What is DirectoryInfo in C#?

The DirectoryInfo Class in C# is a class that is available inside the System.IO namespace. The DirectoryInfo class contains almost a similar feature as the FileInfo class of C#. The only difference is that the DirectoryInfo focuses only on the Directory, not the file systems. So, when we talk about the DirectoryInfo class, we talk about the physical directory. With its help, we get the object with which we can create, delete, and also, we can make some subdirectories, and many more operations can be performed.

It provides several methods to perform operations related to directories and subdirectories. It is a sealed class, and hence it cannot be inherited. You will see the following if you go to the definition of DirectoryInfo class.

```
namespace System.IO
{
    ...public sealed class DirectoryInfo : FileSystemInfo
    {
        ...public DirectoryInfo(string path);

        ...public DirectoryInfo Parent { get; }
        ...public override string FullName { get; }
        ...public override string Name { get; }
        ...public override bool Exists { get; }
        ...public DirectoryInfo Root { get; }

        ...public void Create();
        ...public void Create(DirectorySecurity directorySecurity);
        ...public DirectoryInfo CreateSubdirectory(string path);
        ...public DirectoryInfo CreateSubdirectory(string path, DirectorySecurity directorySecurity);
        ...public void Delete(bool recursive);
        ...public override void Delete();
        ...public IEnumerable<DirectoryInfo> EnumerateDirectories();
        ...public IEnumerable<DirectoryInfo> EnumerateDirectories(string searchPattern, SearchOption searchOption);
        ...public IEnumerable<DirectoryInfo> EnumerateDirectories(string searchPattern);
        ...public IEnumerable<FileInfo> EnumerateFiles(string searchPattern, SearchOption searchOption);
        ...public IEnumerable<FileInfo> EnumerateFiles(string searchPattern);
        ...public IEnumerable<FileInfo> EnumerateFiles();
        ...public IEnumerable<FileSystemInfo> EnumerateFileSystemInfos(string searchPattern, SearchOption searchOption);
        ...public IEnumerable<FileSystemInfo> EnumerateFileSystemInfos();
        ...public IEnumerable<FileSystemInfo> EnumerateFileSystemInfos(string searchPattern);
        ...public DirectorySecurity GetAccessControl(AccessControlSections includeSections);
        ...public DirectorySecurity GetAccessControl();
        ...public DirectoryInfo[] GetDirectories();
        ...public DirectoryInfo[] GetDirectories(string searchPattern, SearchOption searchOption);
        ...public DirectoryInfo[] GetDirectories(string searchPattern);
        ...public FileInfo[] GetFiles();
        ...public FileInfo[] GetFiles(string searchPattern, SearchOption searchOption);
        ...public FileInfo[] GetFiles(string searchPattern);
        ...public FileSystemInfo[] GetFileSystemInfos(string searchPattern, SearchOption searchOption);
        ...public FileSystemInfo[] GetFileSystemInfos();
        ...public FileSystemInfo[] GetFileSystemInfos(string searchPattern);
        ...public void MoveTo(string destDirName);
        ...public void SetAccessControl(DirectorySecurity directorySecurity);
        ...public override string ToString();
    }
}
```

The DirectoryInfo class in C# provides the following constructor, methods, and properties to work with the directory and subdirectory.

The Constructor of DirectoryInfo Class in C#

The DirectoryInfo class provides the following Constructor

public DirectoryInfo(string path): It initializes a new instance of the DirectoryInfo class on the specified path. Here, the variable path specifies the path to create the DirectoryInfo.

Properties of DirectoryInfo Class in C#

The DirectoryInfo class provides the following properties.

1. **Parent:** It is used to get the parent directory of a specified subdirectory. It returns the parent directory or null if the path is null or if the file path denotes a root (such as "\", "C:", or * "\\server\share").
2. **FullName:** It is used to get the full path of the directory. It returns a string containing the full path.
3. **Name:** It is used to get the name of this System.IO.DirectoryInfo instance. It returns the directory name.
4. **Exists:** It is used to get a value indicating whether the directory exists. It returns true if the directory exists; otherwise, false.
5. **Root:** It is used to get the root portion of the directory. It returns an object that represents the root of the directory.
6. **CreationTime:** It is used to get or set the creation time of the current file or directory. It returns the creation date and time of the current System.IO.FileSystemInfo object.
7. **LastAccessTime:** It is used to get or set the time the current file or directory was last accessed. It returns the time that the current file or directory was last accessed.
8. **LastWriteTime:** It is used to get or set the time when the current file or directory was last written. It returns the time the current file was last written.
9. **Extension:** It is used to get the string representing the extension part of the file. It returns a string containing the System.IO.FileSystemInfo extension.

DirectoryInfo Class Methods in C#

The DirectoryInfo class in C# provides the following methods.

1. **Create():** This method is used to create a directory.
2. **Create(DirectorySecurity directorySecurity):** This method creates a directory using a DirectorySecurity object. The directorySecurity parameter specifies the access control to apply to the directory.
3. **CreateSubdirectory(string path):** This method creates a subdirectory or subdirectory on the specified path. The specified path can be relative to this instance of the DirectoryInfo class. The parameter path specified path.
4. **CreateSubdirectory(string path, DirectorySecurity directorySecurity):** This method creates a subdirectory or subdirectories on the specified path with the specified security. The specified path can be relative to this instance of the DirectoryInfo class. The parameter path specified path. This cannot be a different disk volume or Universal Naming Convention (UNC) name. The directorySecurity parameter specifies the security to apply.
5. **Delete():** This method is used to delete the DirectoryInfo if it is empty.

6. **Delete(bool recursive):** This method is used to delete this instance of a DirectoryInfo, specifying whether to delete subdirectories and files. The recursive parameter specifies true to delete this directory, its subdirectories, and all files; otherwise, it is false.
7. **EnumerateDirectories():** This method returns an enumerable collection of directory information in the current directory. It returns an enumerable collection of directories in the current directory.
8. **EnumerateFiles():** This method returns an enumerable file information collection in the current directory. It returns an enumerable collection of the files in the current directory.
9. **GetAccessControl():** This method is used to get the DirectorySecurity object that encapsulates the access control list (ACL) entries for the directory described by the current DirectoryInfo object. This method returns a DirectorySecurity object that encapsulates the access control rules for the directory.
10. **GetDirectories():** This method returns the subdirectories of the current directory. It returns an array of System.IO.DirectoryInfo objects.
11. **GetFiles():** This method returns a file list from the current directory. It returns an array of type System.IO.FileInfo.
12. **MoveTo(string destDirName):** This method moves a DirectoryInfo instance and its contents to a new path. The parameter destDirName specifies the name and path to which to move this directory. The destination cannot be another disk volume or a directory with an identical name. It can be an existing directory to which you want to add this directory as a subdirectory.
13. **SetAccessControl(DirectorySecurity directorySecurity):** This method is used to set access control list (ACL) entries described by a DirectorySecurity object. The parameter directorySecurity specifies an object that describes an ACL entry to apply to the directory described by the path parameter.
14. **ToString():** It returns the original path that the user passed.

Create a new directory in C#:

The Create method of the DirectoryInfo is used to create a new directory. In the below example, we are creating a folder called MyTestFile1 in the D Drive.

```
using System;
using System.IO;
namespace DirectoryInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            String path = @"D:\MyTestFile1";
            DirectoryInfo fl = new DirectoryInfo(path);
            fl.Create();
            {
                Console.WriteLine("Directory has been created");
            }
            Console.ReadKey();
        }
    }
}
```

Once you execute the above code, verify the folder in the D drive of your machine.

Create SubDirectory in C#

The CreateSubdirectory method of the DirectoryInfo class creates a subdirectory or subdirectories on the specified path. In the below example, we are creating a folder called MyTestFile2 inside the MyTestFile1 folder.

```
using System;
using System.IO;
namespace DirectoryInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            String path = @"D:\MyTestFile1";
            DirectoryInfo fl = new DirectoryInfo(path);
            DirectoryInfo dis = fl.CreateSubdirectory("MyTestFile2");
            {
                Console.WriteLine("SubDirectory has been created");
            }
            Console.ReadKey();
        }
    }
}
```

Move a Directory in C#

The MoveTo method of the DirectoryInfo class in C# moves a directory to a new location, including its contents. In the below example, we are moving the MyTestFile1 directory inside the NewTestFile1 directory.

```
using System;
using System.IO;
namespace DirectoryInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            String path1 = @"D:\MyTestFile1";
            string path2 = @"D:\NewTestFile1";
            DirectoryInfo directoryInfo1 = new DirectoryInfo(path1);
            DirectoryInfo directoryInfo2 = new DirectoryInfo(path2);
            directoryInfo1.MoveTo(path2);
            {
                Console.WriteLine("Directory has been Moved");
            }

            Console.ReadKey();
        }
    }
}
```

Delete a directory in C#

The Delete method of the DirectoryInfo class in C# is used to delete a directory, specifying whether to delete subdirectories and files. In the below example, we are deleting the NewTestFile1 directory from the D drive.

```
using System;
using System.IO;
namespace DirectoryInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string path = @"D:\NewTestFile1 ";
            DirectoryInfo directoryInfo1 = new DirectoryInfo(path);
            directoryInfo1.Delete();
            {
                Console.WriteLine("Directory has been deleted");
            }
            Console.ReadKey();
        }
    }
}
```

How do I get directory details in C#?

The following example shows the use of the DirectoryInfo class in C#. In the below example, we check for a directory "D:\Dotnet" and if then the directory will be there in the system. it will show the directory information else it will create a new directory, D:\Dotnet.

```
using System;
using System.IO;
namespace DirectoryInfoDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string DirectoryPath = @"D:\Dotnet";
            DirectoryInfo directoryInfo = new DirectoryInfo(DirectoryPath);
            try
            {
                if (directoryInfo.Exists)
                {
                    Console.WriteLine("{0} Directory is already exists", Direc
                    Console.WriteLine("Directory Name : " + directoryInfo.Name
                    Console.WriteLine("Path : " + directoryInfo.FullName);
                    Console.WriteLine("Directory is created on : " +
                    directoryInfo.CreationTime);
                }
            }
        }
    }
}
```

```

        Console.WriteLine("Directory is Last Accessed on " +
directoryInfo.LastAccessTime);
    }
    else
    {
        directoryInfo.Create();
        Console.WriteLine(DirectoryPath + "Directory is created
successfully");
    }
    //Delete this directory
    Console.WriteLine("If you want to delete this directory press
Press any key to exit.");
    try
    {
        char ch = Convert.ToChar(Console.ReadLine());
        if (ch == 'y')
        {
            if (directoryInfo.Exists)
            {
                directoryInfo.Delete();
                Console.WriteLine(DirectoryPath + "Directory Delet
            }
            else
            {
                Console.WriteLine(DirectoryPath + "Directory Not f
            }
        }
    }
    catch
    {
        Console.WriteLine("Press Enter to Exit");
    }
    Console.ReadKey();
}
catch (DirectoryNotFoundException d)
{
    Console.WriteLine("Exception raised : " + d.Message);
    Console.ReadKey();
}
}
}
}
}

```

Once you run the above code, you will get the following output. Notice that this directory does not exist, and hence it will create the directory.

```

D:\DotnetDirectory is created successfully
If you want to delete this directory press small y. Press any key to exit.

```

Now, press any key except y, close the program, and run again. As the directory was already created in the first run, you will get the following output this time.

```
D:\Dotnet Directory is already exists
Directory Name : Dotnet
Path : D:\Dotnet
Directory is created on : 06-05-2022 15:41:42
Directory is Last Accessed on 06-05-2022 15:41:42
If you want to delete this directory press small y. Press any key to exit.
```

In the next article, I will discuss [How to work with Excel Files in C#](#) with Examples. In this article, I try to explain the **DirectoryInfo Class in C#** with Examples. I hope you enjoy this DirectoryInfo Class in C# with Examples article. I would like to have your feedback. Please post your feedback, questions, or comments about this article.